

Scala

основы языка программирования

Первая презентация: запуск и базовые конструкции языка

Команда 4. Бурачевский,
Киселева, Ятманова.
Группа 5030102/30202

Scala в одном слайде

Минимум истории — только то, что помогает понять язык

Что это за язык

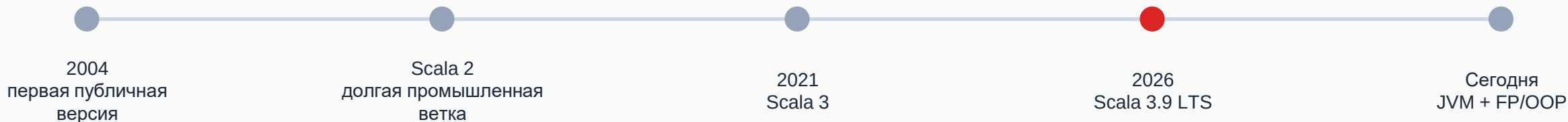
Scala = Scalable Language
Мультипарадигменный язык: объектный + функциональный стиль
Статическая типизация и вывод типов

Где выполняется

Основной runtime — JVM
Scala-код компилируется в JVM bytecode
Можно использовать Java-библиотеки

Версия для примеров

В презентации используем Scala 3
Практически весь базовый код работает и в Scala 3.8/3.9
Синтаксис основан на Scala 3



Как запускать Scala-программу

Основной вариант для нашего проекта — IntelliJ IDEA; под капотом всё выполняется на JVM

В IntelliJ IDEA

1. JDK 21
2. Scala plugin
3. Build system: IntelliJ
4. Scala SDK 3.x
5. Файл .scala → зелёная кнопка Run

```
@main
def hello(): Unit =
  println("Hello, Scala!")
```

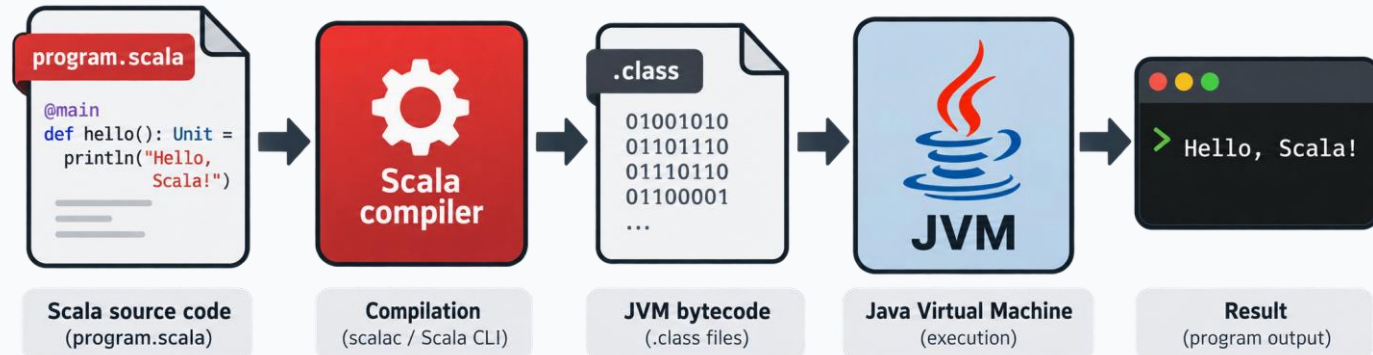
Что происходит после Run:

Hello World

Создаем Main.scala и запускаем функцию, помеченную @main.

После Run IDEA компилирует код и передает его JVM.

Альтернативы: Scala CLI · Scastie · Docker



Примеры без сторонних библиотек запускаются через встроенную сборку IDEA.

Структура простой программы

Разбираем минимальный Scala-файл и правила синтаксиса

```
@main
def main(): Unit =
  println("Hello, Scala!")
```

Минимальная программа = функция,
помеченная `@main` + тело функции.

```
if x > 0 then
  println("positive")
else
  println("negative")
```

@main

аннотация точки входа: именно эту функцию можно запускать как программу

def main()

объявление функции main без входных параметров

Unit

тип результата, аналог “ничего полезного не возвращает”

отступы

в Scala 3 блоки можно задавать отступами, а не фигурными скобками

В Scala 3 часто используется “новый” синтаксис: `then/do/отступы`. Он делает код ближе к Python, но типизация остаётся статической.

Один файл может содержать несколько функций, но запуск начинается с `@main`

Переменные и типы

Scala статически типизирована, но умеет выводить типы автоматически

```
val language = "Scala"

var version = 3
version = 4

val x = 10
x = 20 // compile error
```

Type inference: тип можно не писать, если компилятор однозначно его понимает.

```
val count = 10 // Int
val title = "Scala" // String
```

val

Неизменяемое значение. После присваивания нельзя переназначить. В Scala используется чаще.

var

Изменяемая переменная. Можно менять значение, но лучше использовать только когда это реально нужно.

Базовые типы:

```
val age: Int = 21
val price: Double = 19.99
val active: Boolean = true
val letter: Char = 'S'
val name: String = "Scala"
```

Операции, строки и преобразование типов

База, которая понадобится в любой программе

Арифметика и сравнения

Обычные операции работают как ожидается: +, -, *, /, %, >, ==, !=.
Важно: `a / b` для `Int` даёт целочисленный результат.

```
val a = 10
val b = 3

a + b // 13
a / b // 3
a % b // 1
a > b // true
```

Интерполяция строк

Перед строкой ставим `s`, а значения вставляем через `$name` или `${expression}`.
Это читается лучше, чем конкатенация.

```
val name = "Anna"
val age = 21

println(s"$name is $age")
println(s"Next year: ${age + 1}")
```

Преобразования

Типы часто приводятся явно методами `.toInt`, `.toDouble`, `.toString`. Ошибочный ввод может упасть во время выполнения.

```
"42".toInt      // 42
10.toDouble     // 10.0
123.toString    // "123"

"hello".toInt   // ошибка
```

if: условие как выражение

В Scala многие конструкции возвращают значение

```
if age >= 18 then  
  println("Adult")  
else  
  println("Minor")
```

Обычное использование: выполнить разные действия в зависимости от условия.

```
val discount =  
  if isStudent then 0.15  
  else 0.0  
  
val finalPrice = price * (1 - discount)
```

```
val status =  
  if age >= 18 then "Adult"  
  else "Minor"
```

Главная особенность

if возвращает значение. Поэтому результат можно сразу записать в val без временной var-переменной.

Scala-style: меньше изменяемого состояния, больше вычисляемых выражений.

match и pattern matching

Похож на switch, но в Scala намного мощнее

```
val message =  
  code match  
    case 200 => "OK"  
    case 404 => "Not Found"  
    case 500 => "Server Error"  
    case _   => "Unknown"
```

Pattern matching в полной силе умеет разбирать не только числа и строки, но и структуру объектов — это будет полезно во второй презентации.

На практике match часто заменяет цепочку if / else if / else

Что важно

match тоже возвращает значение. Каждая строка case описывает вариант сопоставления. Символ _ означает "любой другой случай".

```
val category =  
  age match  
    case x if x < 18 => "Minor"  
    case x if x < 65 => "Adult"  
    case _           => "Senior"
```


Циклы и диапазоны

В Scala есть привычные циклы, но стиль языка чаще ведёт к коллекциям и итераторам

```
for i <- 1 to 5 do  
  println(i)
```

1 to 5

Диапазон с включённой правой границей: 1, 2, 3, 4, 5

```
for i <- 1 until 5 do  
  println(i)
```

1 until 5

Диапазон без правой границы: 1, 2, 3, 4

```
var i = 0
```

```
while i < 5 do  
  println(i)  
  i += 1
```

while

Нужна изменяемая переменная var.
Используется, когда заранее не удобен for
или нужно явно управлять состоянием.

В Scala for работает не только с диапазонами, но и с коллекциями:

```
val names = List("Ann", "Bob", "Max")  
  
for name <- names do  
  println(s"Hello, $name")
```

Итераторы

Iterator — объект для последовательного прохода по элементам

```
val numbers = List(10, 20, 30)
val it = numbers.iterator
```

```
while it.hasNext do
  println(it.next())
```

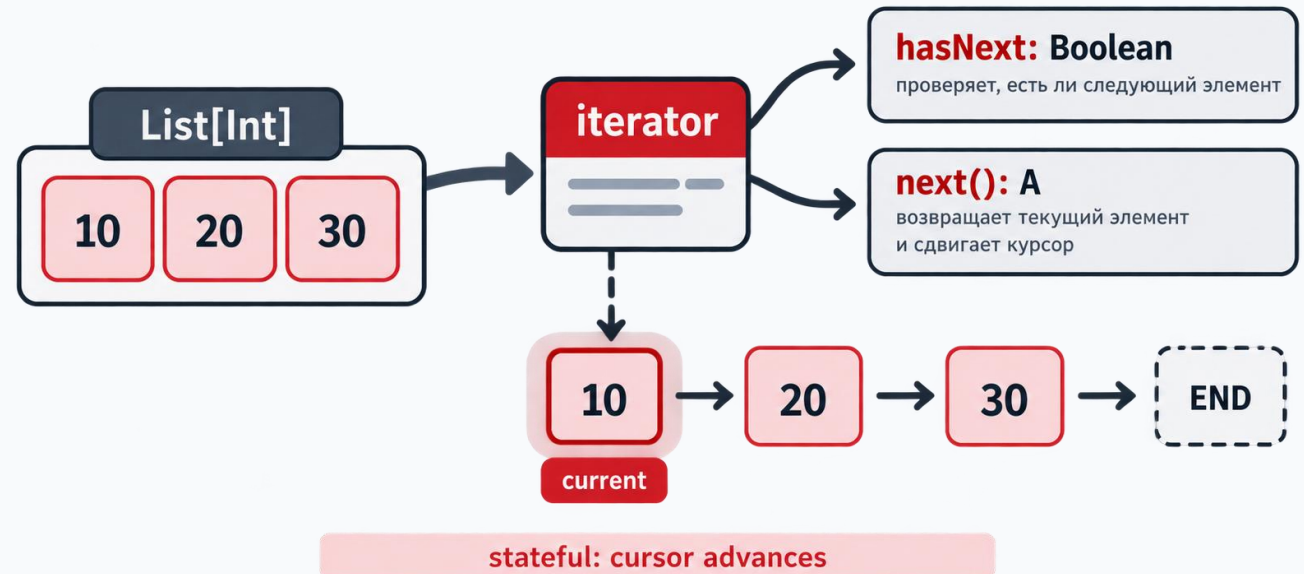
hasNext

Возвращает Boolean: есть ли следующий элемент?

next()

Возвращает текущий элемент и сдвигает курсор дальше.

Важно: Iterator хранит состояние. После прохода он “израсходован”; чтобы пройти заново, обычно создают новый iterator.



Итераторы напрямую связаны с for, foreach и чтением файла построчно

Основные коллекции

Базовые контейнеры, с которыми удобно писать небольшие программы

```
val numbers = List(1, 2, 3)
```

List

Упорядоченная последовательность.
Часто используется как неизменяемая коллекция.

```
val ids = Set(10, 20, 30)
```

Set

Множество: хранит уникальные элементы, порядок не является основной идеей.

```
val data = Array(1, 2, 3)  
data(0) = 100
```

Array

Массив фиксированного размера.
Элементы можно изменять по индексу.

```
val user = Map(  
  "name" -> "Anna",  
  "city" -> "SPb"  
)
```

Map

Хранит пары ключ → значение.
Удобно для словарей и справочников.

Практическое правило Scala: сначала выбираем immutable collection, mutable используем только если нужна явная изменяемость.

Работа с коллекциями: map, filter, foreach

Три метода, которые сразу показывают функциональный стиль Scala

```
val numbers = List(1, 2, 3, 4, 5)
```

map

Преобразует каждый элемент и возвращает новую коллекцию.

```
numbers.map(x => x * x)  
// List(1, 4, 9, 16, 25)
```

filter

Оставляет только элементы, которые прошли условие.

```
numbers.filter(x => x % 2 == 0)  
// List(2, 4)
```

foreach

Выполняет действие для каждого элемента, обычно без создания новой коллекции.

```
numbers.foreach(x => println(x))  
// печатает элементы
```

Короткая форма lambda:

```
numbers.map(_ * 2)
```

Не путать: Map — коллекция ключ→значение, а .map() — метод преобразования элементов.

Функции и lambda

Функции можно объявлять, возвращать значения и передавать как данные

```
def square(x: Int): Int =  
  x * x  
  
val result = square(5)
```

Как читается сигнатура

def — объявление функции
square — имя
x: Int — параметр
: Int — тип результата

return обычно не нужен

Результатом функции становится последнее выражение в теле.

```
val squareFun = (x: Int) => x * x  
  
val doubled = List(1, 2, 3).map(x => x * 2)  
  
val short = List(1, 2, 3).map(_ * 2)
```

Lambda — безымянная функция. Её можно сохранить в переменную или передать в map/filter/foreach.

Области видимости

Значение доступно внутри того блока, где оно объявлено

```
val x = 10

def example(): Unit =
  val y = 20

  if x > 0 then
    val z = 30
    println(x + y + z)

  // println(z) // ошибка
```

global scope

function scope

if block scope

Правило

x видна внутри функции; y — только внутри example; z — только внутри блока if.

Ввод и вывод в консоль

Стандартный минимум: `print/println` + `scala.io.StdIn.readLine`

```
import scala.io.StdIn.readLine

@main
def consoleDemo(): Unit =
  print("Enter your name: ")
  val name = readLine()

  print("Enter your age: ")
  val age = readLine().toInt

  println(s"Hello, $name!")
  println(s"Next year: ${age + 1}")
```

```
Enter your name: Anna
Enter your age: 21
```

```
Hello, Anna!
Next year: 22
```

Что важно

`readLine()` всегда возвращает `String`. Если нужно число, используем `.toInt`, но некорректный ввод может дать ошибку.

Чтение файла

Файл можно читать целиком или построчно; построчное чтение возвращает `Iterator[String]`

```
import scala.io.Source

val source = Source.fromFile("numbers.txt")

for line <- source.getLines() do
  println(line)

source.close()
```

numbers.txt

10
20
30
40



getLines()

Возвращает `Iterator[String]`: строки выдаются по одной, а не обязательно хранятся все сразу.

close()

После работы файл нужно закрыть, чтобы освободить системный ресурс.

Почему это связано с Iterator

Файл может быть большим, поэтому удобнее получать строки постепенно.

Запись в файл

Scala может напрямую использовать классы стандартной библиотеки Java

```
import java.io.PrintWriter

val writer = new PrintWriter("result.txt")

writer.println("Hello, Scala!")
writer.println("Second line")

writer.close()
```

Scala program



result.txt

PrintWriter

Простой способ записать строки в файл. После записи вызываем close().

Java interoperability

Scala работает на JVM, поэтому может использовать java.io, java.time и другие Java API.

Итоговый пример: чтение → обработка → запись

Одна программа объединяет val, Iterator, map/filter, преобразования типов и файлы

```
import scala.io.Source
import java.io.PrintWriter

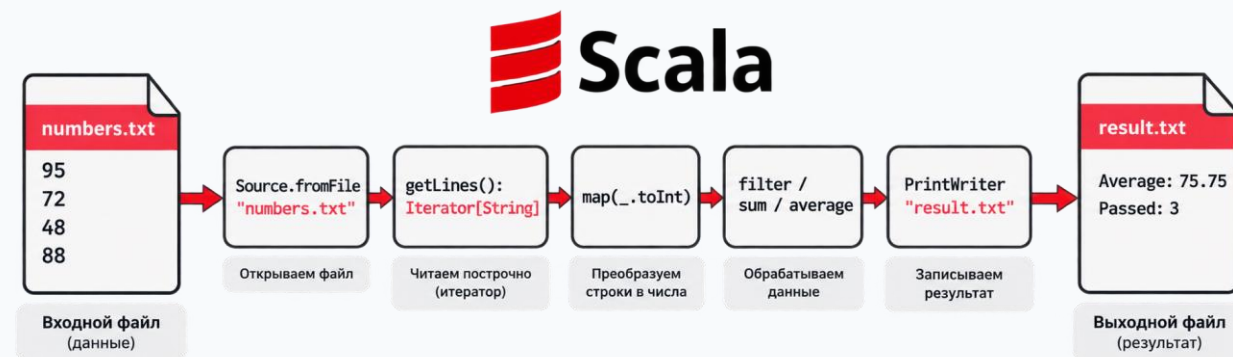
@main
def main(): Unit =
  val source = Source.fromFile("numbers.txt")

  val scores =
    source.getLines().map(_.toInt).toList

  source.close()

  val passed = scores.filter(_ >= 60)
  val average = scores.sum.toDouble / scores.length

  val writer = new PrintWriter("result.txt")
  writer.println(s"Average: $average")
  writer.println(s"Passed: ${passed.length}")
  writer.close()
```



val

Iterator

map

filter

toInt

PrintWriter

Файл читается построчно как `Iterator[String]`, строки превращаются в числа, затем результат записывается в новый файл.

Итоги первой презентации

Что уже умеем после базового знакомства со Scala

запустить программу

понимать JVM-схему

объявлять `val / var`

работать с типами

писать `if` и `match`

использовать `for / while`

понимать `Iterator`

работать с `List / Set / Map`

использовать `map/filter`

писать функции

читать консоль

читать и писать файлы

Что рассмотрим на следующих презентациях:

классы, объекты, `traits` и `case classes`

рекурсия, замыкания и функции высшего порядка

`generics`, `packages/import/export`

ошибки, конкурентность, JVM и память

Вопросы?